

Universidade Presbiteriana Mackenzie

Ciência da Computação
Laboratório de Engenharia de Software

E-Commerce TDV

Documentação Final do Projeto

Integrantes:

Maurício Custódio Vicentini

Victor Sung Woo Hong

São Paulo

2026

Sumário

1	Introdução	6
2	Definição da Demanda	7
2.1	Problema ou Oportunidade Percebida	7
2.2	Razão e Justificativa	7
2.3	Descrição do Produto de Software	7
2.4	Stakeholders	8
2.5	Principais Etapas de Desenvolvimento	8
2.6	CrITÉrios de Qualidade	9
3	Escopo do Projeto	9
3.1	Escopo Inicial Planejado	9
3.2	Escopo Implementado no MVP	10
3.3	Funcionalidades Fora do Escopo Final	11
3.4	Justificativa do Escopo Entregue	12
3.5	Melhorias Futuras	12
3.6	Conclusão do Escopo	13
4	Requisitos do Produto	13
4.1	Requisitos Funcionais	13
4.2	Requisitos Não Funcionais	15
4.3	Backlog do Produto	17
4.4	Itens Planejados para Evolução	18
4.5	Priorização das Tarefas	18
4.6	Considerações sobre os Requisitos	19
5	Arquitetura da Solução	19
5.1	Visão Geral da Arquitetura	19
5.2	Separação entre Front-end e Back-end	20

5.3	Comunicação via API REST	20
5.4	Camadas da Aplicação	21
5.5	Arquitetura do Back-end	21
5.6	Arquitetura do Front-end	22
5.7	Arquitetura de Deploy	22
5.8	CORS	23
5.9	Justificativa da Arquitetura Escolhida	23
6	Estrutura do Back-end	24
6.1	Visão Geral	24
6.2	Organização em src/app	24
6.3	Camada db	25
6.4	Camada schemas	25
6.5	Camada api	25
6.6	Banco de Dados	26
6.7	Seed de Dados	27
6.8	Funcionamento Geral do Back-end	27
6.9	Justificativa da Estrutura	27
7	Estrutura do Front-end	28
7.1	Visão Geral	28
7.2	React e Vite	28
7.3	Pasta src	29
7.4	Pasta public	29
7.5	Estados no React	29
7.6	Consumo da API	30
7.7	Busca e Filtro	31
7.8	Carrinho Local	31
7.9	Área Administrativa	31
7.10	Build e Publicação	32
7.11	Justificativa da Estrutura	32

8	Funcionalidades Implementadas	32
8.1	Visão Geral	32
8.2	Catálogo de Produtos	33
8.3	Categorias	33
8.4	Busca por Produto	33
8.5	Filtro por Categoria	33
8.6	Carrinho Local	34
8.7	Área Administrativa Simples	34
8.8	Cadastro de Produtos	34
8.9	Edição de Produtos	35
8.10	Exclusão de Produtos	35
8.11	Imagens Reais dos Produtos	35
8.12	Integração com API	36
8.13	Deploy da Aplicação	36
8.14	Considerações sobre as Funcionalidades	36
9	Modelagem do Sistema	36
9.1	Visão Geral da Modelagem	36
9.2	Casos de Uso	37
9.3	Descrição dos Principais Casos de Uso	38
9.4	Diagrama de Domínio	38
9.5	Diagramas de Sequência	39
9.6	Diagrama de Arquitetura	39
9.7	Diagrama de Implantação	40
9.8	Considerações sobre a Modelagem	41
10	Deploy em Nuvem	41
10.1	Visão Geral	41
10.2	AWS Academy Learner Lab	41
10.3	Instância EC2	42
10.4	Security Group	42

10.5	Deploy do Back-end	42
10.6	Deploy do Front-end	43
10.7	Nginx	43
10.8	Comunicação entre Front-end e Back-end	44
10.9	CORS	44
10.10	Imagens no Ambiente Cloud	44
10.11	Limitações do Deploy	45
10.12	Considerações sobre o Deploy	45
11	Testes e Validações	45
11.1	Visão Geral	45
11.2	Testes do Back-end	46
11.3	Testes do Front-end	46
11.4	Testes da Área Administrativa	47
11.5	Testes de Integração	48
11.6	Testes de Deploy	48
11.7	Validação do Carrinho Local	49
11.8	Validação das Imagens dos Produtos	49
11.9	Problemas Encontrados e Soluções	49
11.10	Considerações sobre os Testes	50
12	Limitações do Protótipo	50
12.1	Visão Geral	50
12.2	Login Administrativo Simplificado	51
12.3	Rotas Administrativas sem Proteção Real	51
12.4	Carrinho Local sem Pedido no Banco	51
12.5	Ausência de Checkout	51
12.6	Ausência de Pagamento Online	52
12.7	Banco SQLite em Ambiente Cloud	52
12.8	IP Público Variável no AWS Academy	52
12.9	Ausência de HTTPS	52

12.10	Responsividade Limitada	53
12.11	Considerações sobre as Limitações	53
13	Melhorias Futuras	53
13.1	Visão Geral	53
13.2	Autenticação Real	53
13.3	Proteção de Rotas Administrativas	54
13.4	Checkout	54
13.5	Registro de Pedidos	54
13.6	Atualização Automática de Estoque	55
13.7	Pagamento Online	55
13.8	Painel Administrativo Completo	55
13.9	Migração do Banco de Dados	55
13.10	Domínio Próprio e HTTPS	56
13.11	Melhorias de Interface e Responsividade	56
13.12	Testes Automatizados	56
13.13	Considerações sobre as Melhorias Futuras	56
14	Conclusão	57
15	Referências Técnicas	57
15.1	Tecnologias de Back-end	58
15.2	Tecnologias de Front-end	58
15.3	Infraestrutura e Deploy	58
15.4	Materiais de Apoio	59
15.5	Considerações sobre as Referências	59

1. Introdução

O presente projeto consiste no desenvolvimento de uma aplicação web de e-commerce simplificada para a **TDV Elétrica**, voltada à apresentação e gerenciamento de produtos do segmento de materiais elétricos. A proposta surgiu a partir da necessidade de criar uma solução digital capaz de organizar o catálogo de produtos, permitir a consulta de informações pelos clientes e oferecer uma base inicial para a evolução futura de uma loja virtual completa.

A aplicação foi desenvolvida com uma arquitetura full-stack, separando o sistema em front-end, back-end e banco de dados. O front-end é responsável pela interface visual e pela interação com o usuário, enquanto o back-end fornece uma API REST para consulta, cadastro, edição e exclusão de dados. O banco de dados armazena as informações referentes aos produtos, categorias e usuários.

O back-end foi implementado em **Python**, utilizando o framework **FastAPI**. Para a persistência dos dados, foi utilizado o banco **SQLite**, com integração por meio do **SQLAlchemy**. A validação dos dados da API foi realizada com **Pydantic**, garantindo maior consistência nas informações recebidas e retornadas pelo sistema.

O front-end foi desenvolvido com **React** e **Vite**, permitindo a construção de uma interface dinâmica e integrada à API. A aplicação permite visualizar produtos, buscar itens por nome, filtrar por categoria, visualizar imagens reais dos produtos e adicionar itens a um carrinho local. Também foi criada uma área administrativa simples, acessada por senha, que permite cadastrar, editar e excluir produtos do catálogo.

Além do desenvolvimento local, o sistema foi publicado em ambiente de nuvem utilizando o **AWS Academy Learner Lab**. A aplicação foi implantada em uma instância **EC2 Ubuntu**, com o back-end executado por **Uvicorn** e mantido em segundo plano com **systemd**. O front-end foi buildado com Vite e servido pelo **Nginx** na porta 80.

O projeto foi desenvolvido no contexto da disciplina de **Laboratório de Engenharia de Software**, com o objetivo de aplicar conceitos de levantamento de requisitos, modelagem, desenvolvimento full-stack, integração entre camadas, versionamento com GitHub e deploy em ambiente cloud.

Por se tratar de um MVP acadêmico, algumas funcionalidades foram implementadas de forma simplificada, como o carrinho local e o login de administrador no front-end. Funcionalidades mais avançadas, como checkout real, geração de pedidos, autenticação segura com token, criptografia de senhas e integração com pagamento, foram definidas como melhorias futuras.

Dessa forma, o projeto entrega uma primeira versão funcional de um e-commerce

para materiais elétricos, contemplando catálogo de produtos, integração com API, banco de dados, carrinho local, área administrativa simples, imagens reais e publicação em nuvem.

2. Definição da Demanda

2.1. Problema ou Oportunidade Percebida

A TDV Elétrica possui a necessidade de organizar e apresentar seus produtos de forma digital, oferecendo aos clientes uma experiência mais prática para consulta de itens, preços, categorias, estoque e imagens. Antes da aplicação, a exposição dos produtos dependia de processos mais manuais, sem um catálogo centralizado e acessível pela internet.

A oportunidade percebida foi desenvolver uma plataforma web que funcionasse como um e-commerce simplificado, permitindo ao cliente visualizar os produtos disponíveis e ao administrador gerenciar o catálogo de forma mais organizada.

2.2. Razão e Justificativa

A criação do sistema se justifica pela necessidade de profissionalizar a apresentação dos produtos da empresa e criar uma base inicial para um futuro e-commerce completo. A aplicação permite centralizar dados importantes, como nome, descrição, preço, estoque, categoria e imagem dos produtos.

Além disso, o projeto atende ao objetivo acadêmico da disciplina de Laboratório de Engenharia de Software, pois envolve levantamento de requisitos, desenvolvimento full-stack, integração entre front-end e back-end, versionamento e deploy em nuvem.

2.3. Descrição do Produto de Software

O produto desenvolvido é uma aplicação web de e-commerce simplificada para materiais elétricos. O sistema permite a visualização de produtos, busca por nome, filtro por categoria, exibição de imagens reais, consulta de preço e estoque, além da utilização de um carrinho local.

A aplicação também possui uma área administrativa simples, acessada por senha, que permite cadastrar, editar e excluir produtos. O sistema foi desenvolvido com back-end em Python com FastAPI, banco SQLite com SQLAlchemy e front-end em React com Vite.

2.4. Stakeholders

Os principais stakeholders do projeto são:

- **Cliente do projeto:** TDV Elétrica / Daniel Viana, responsável pela necessidade de digitalização do catálogo.
- **Usuários finais:** clientes que acessam o site para consultar produtos e simular uma compra.
- **Administrador:** usuário responsável por cadastrar, editar e excluir produtos no catálogo.
- **Desenvolvedores:** grupo responsável pela análise, implementação, testes, deploy e documentação.
- **Professor/orientador:** responsável pela avaliação acadêmica do projeto.

2.5. Principais Etapas de Desenvolvimento

O desenvolvimento do projeto foi organizado nas seguintes etapas:

1. Planejamento e definição da demanda.
2. Levantamento de requisitos e modelagem inicial.
3. Desenvolvimento do back-end com FastAPI.
4. Criação do banco de dados com SQLite e SQLAlchemy.
5. Desenvolvimento do front-end com React e Vite.
6. Integração entre front-end e back-end via API REST.
7. Implementação do carrinho local e da área administrativa simples.
8. Inserção de imagens reais dos produtos.
9. Versionamento do projeto no GitHub.
10. Deploy em ambiente cloud utilizando AWS EC2.
11. Testes e validação final da aplicação.

2.6. Critérios de Qualidade

Os principais critérios de qualidade definidos para o MVP foram:

- **Usabilidade:** a interface deve ser simples e permitir navegação clara pelo catálogo.
- **Organização visual:** os produtos devem ser exibidos com imagem, nome, descrição, preço, categoria e estoque.
- **Integração:** o front-end deve consumir dados reais da API FastAPI.
- **Persistência:** os dados dos produtos e categorias devem ser armazenados no banco SQLite.
- **Manutenibilidade:** o código deve ser organizado em camadas e pastas bem definidas.
- **Carrinho local:** os itens adicionados ao carrinho devem permanecer salvos no navegador via `localStorage`.
- **Controle administrativo:** ações de cadastro, edição e exclusão devem aparecer apenas no modo administrador.
- **Deploy:** a aplicação deve estar acessível em ambiente cloud por meio de uma instância EC2.

Dessa forma, a demanda do projeto foi definida a partir da necessidade de criar uma solução digital para apresentação e gerenciamento de produtos elétricos. O MVP desenvolvido entrega um catálogo funcional integrado a uma API, com carrinho local, área administrativa simples, imagens reais e deploy em nuvem.

3. Escopo do Projeto

3.1. Escopo Inicial Planejado

O escopo inicial do projeto previa o desenvolvimento de uma plataforma de e-commerce para a TDV Elétrica, contemplando funcionalidades relacionadas à visualização de produtos, carrinho de compras, cadastro de usuários, login, finalização de pedidos e área administrativa.

A proposta inicial buscava simular o funcionamento de uma loja virtual completa, permitindo que o cliente navegasse pelo catálogo, adicionasse produtos ao carrinho e

concluísse uma compra. Também foi prevista uma área administrativa para gerenciamento de produtos e controle do catálogo.

Durante o desenvolvimento, o escopo foi ajustado para a construção de um MVP funcional, priorizando as funcionalidades essenciais para apresentação do catálogo, integração entre front-end e back-end, gerenciamento de produtos e publicação em nuvem.

3.2. Escopo Implementado no MVP

O MVP desenvolvido teve como foco principal a criação de uma aplicação funcional para consulta e gerenciamento de produtos elétricos.

As funcionalidades implementadas foram:

1. Catálogo de produtos elétricos.
2. Listagem de produtos cadastrados.
3. Exibição de nome, descrição, preço, estoque, categoria e imagem.
4. Utilização de imagens reais dos produtos.
5. Consulta de categorias.
6. Filtro de produtos por categoria.
7. Busca de produtos por nome.
8. Carrinho local com persistência no navegador.
9. Adição, remoção e alteração de quantidade de itens no carrinho.
10. Cálculo do total do carrinho.
11. Área administrativa simples.
12. Login simples de administrador no front-end.
13. Cadastro de novos produtos.
14. Edição de produtos existentes.
15. Exclusão de produtos.
16. CRUD de produtos no back-end.
17. API de categorias.

18. Estrutura de usuários no back-end.
19. Integração entre React e FastAPI via API REST.
20. Banco de dados SQLite.
21. Deploy em AWS EC2.
22. Front-end servido com Nginx.
23. Back-end executado com Uvicorn e systemd.

Com isso, o sistema entregue permite que usuários consultem produtos e simulem uma compra por meio do carrinho local, enquanto o administrador consegue gerenciar o catálogo de produtos.

3.3. Funcionalidades Fora do Escopo Final

Algumas funcionalidades previstas inicialmente não foram implementadas na versão final do MVP, pois exigiriam maior complexidade técnica e mais tempo de desenvolvimento.

As principais funcionalidades fora do escopo final foram:

1. Checkout real.
2. Geração de número de pedido.
3. Persistência de pedidos no banco de dados.
4. Pagamento online.
5. Integração com gateway de pagamento.
6. Login real de usuários com autenticação segura.
7. Token JWT.
8. Criptografia de senhas.
9. Histórico de compras.
10. Rastreamento de pedidos.
11. Baixa automática de estoque após compra.
12. Cupons de desconto.

13. Envio de e-mail de confirmação.

Essas funcionalidades foram mantidas como melhorias futuras, pois dependem de uma estrutura mais completa de autenticação, pedidos, pagamentos e regras de negócio.

3.4. Justificativa do Escopo Entregue

A decisão de concentrar o desenvolvimento no catálogo, carrinho local, área administrativa simples e deploy foi tomada para garantir a entrega de uma aplicação funcional, estável e demonstrável dentro do prazo disponível.

O foco do projeto foi validar a arquitetura full-stack e a comunicação entre as principais camadas do sistema: front-end React, back-end FastAPI, banco SQLite, API REST e deploy em AWS EC2.

Essa escolha permitiu desenvolver uma base sólida para o sistema, com funcionalidades suficientes para demonstrar o funcionamento de um e-commerce simplificado. Além disso, o escopo implementado contempla aspectos importantes de engenharia de software, como organização de código, separação de responsabilidades, versionamento, integração entre sistemas, documentação e implantação em nuvem.

3.5. Melhorias Futuras

Como evolução do projeto, podem ser implementadas as seguintes melhorias:

1. Autenticação real no back-end.
2. Login com e-mail e senha.
3. Criptografia de senhas.
4. Token JWT para proteger rotas administrativas.
5. Tabela de pedidos.
6. Checkout simulado ou real.
7. Registro de pedidos no banco de dados.
8. Atualização automática do estoque após compra.
9. Histórico de pedidos para o cliente.
10. Painel administrativo separado.

11. Controle de permissões.
12. Migração do banco SQLite para PostgreSQL ou MySQL.
13. Uso de banco em nuvem, como Amazon RDS.
14. Configuração de domínio próprio.
15. Configuração de HTTPS.
16. Melhorias de responsividade para dispositivos móveis.

Essas melhorias aproximariam o sistema de um e-commerce real de produção, aumentando segurança, escalabilidade e robustez.

3.6. Conclusão do Escopo

O escopo final do projeto foi definido com base na entrega de um MVP funcional. Embora nem todas as funcionalidades previstas inicialmente tenham sido implementadas, o sistema entregue atende ao objetivo principal de criar uma plataforma de catálogo e gerenciamento de produtos para a TDV Elétrica.

A aplicação permite visualizar produtos, utilizar busca e filtro, adicionar itens ao carrinho, gerenciar produtos em modo administrador e acessar o sistema em ambiente cloud. Dessa forma, o projeto cumpre seu papel acadêmico e técnico, demonstrando a construção de uma aplicação full-stack integrada e publicada em nuvem.

4. Requisitos do Produto

4.1. Requisitos Funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve oferecer aos usuários. No projeto E-Commerce TDV, os requisitos foram definidos com base no escopo de um MVP para catálogo e gerenciamento de produtos elétricos.

ID	Requisito Funcional	Descrição	Prioridade	Status
RF01	Listar produtos	O sistema deve listar os produtos cadastrados, exibindo nome, descrição, preço, estoque, categoria e imagem.	Alta	Implementado
RF02	Consultar categorias	O sistema deve permitir a consulta das categorias cadastradas.	Alta	Implementado
RF03	Filtrar produtos por categoria	O sistema deve permitir filtrar os produtos conforme a categoria selecionada.	Alta	Implementado
RF04	Buscar produtos por nome	O sistema deve permitir a busca de produtos pelo nome.	Alta	Implementado
RF05	Exibir imagens dos produtos	O sistema deve exibir imagens reais associadas aos produtos.	Alta	Implementado
RF06	Adicionar produtos ao carrinho	O usuário deve conseguir adicionar produtos ao carrinho.	Alta	Implementado
RF07	Remover produtos do carrinho	O usuário deve conseguir remover itens do carrinho.	Alta	Implementado
RF08	Alterar quantidade no carrinho	O usuário deve conseguir aumentar ou diminuir a quantidade dos itens no carrinho.	Média	Implementado
RF09	Persistir carrinho localmente	O sistema deve manter os itens do carrinho salvos no navegador por meio de <code>localStorage</code> .	Média	Implementado
RF10	Calcular total do carrinho	O sistema deve calcular automaticamente o total dos itens adicionados ao carrinho.	Média	Implementado
RF11	Acessar área administrativa	O sistema deve permitir acesso a uma área administrativa simples por meio de senha.	Média	Implementado
RF12	Cadastrar produtos	O administrador deve conseguir cadastrar novos produtos no catálogo.	Alta	Implementado

ID	Requisito Funcional	Descrição	Prioridade	Status
RF13	Editar produtos	O administrador deve conseguir editar informações de produtos existentes.	Alta	Implementado
RF14	Excluir produtos	O administrador deve conseguir excluir produtos do catálogo.	Média	Implementado
RF15	Ocultar ações administrativas	O sistema deve ocultar os botões de cadastro, edição e exclusão para usuários comuns.	Alta	Implementado
RF16	Criar estrutura de usuários	O sistema deve possuir model, schema e rotas básicas para usuários.	Média	Implementado
RF17	Finalizar compra	O sistema deve permitir finalizar uma compra e gerar um pedido.	Alta	Não implementado
RF18	Autenticação real	O sistema deve permitir login real com credenciais seguras.	Média	Não implementado
RF19	Registrar pedidos	O sistema deve registrar pedidos no banco de dados.	Alta	Não implementado
RF20	Gerenciar cupons	O sistema deve permitir criação e validação de cupons de desconto.	Baixa	Não implementado

4.2. Requisitos Não Funcionais

Os requisitos não funcionais representam características de qualidade, desempenho, segurança e organização do sistema.

ID	Requisito Não Funcional	Descrição	Prioridade	Status
RNF01	Interface responsiva	A interface deve funcionar adequadamente em telas de desktop e notebook.	Alta	Implementado
RNF02	Comunicação HTTP/JSON	A comunicação entre front-end e back-end deve ser feita por requisições HTTP com respostas em JSON.	Alta	Implementado

ID	Requisito Não Funcional	Descrição	Prioridade	Status
RNF03	Separação entre front-end e back-end	O sistema deve manter separação entre interface e API.	Alta	Implementado
RNF04	Organização em camadas	O back-end deve ser organizado em camadas, separando banco, schemas e rotas.	Alta	Implementado
RNF05	Persistência em banco	Os dados devem ser armazenados em banco de dados SQLite.	Alta	Implementado
RNF06	Validação de dados	A API deve validar os dados recebidos por meio de schemas Pydantic.	Alta	Implementado
RNF07	Manutenibilidade	O código deve ser organizado em pastas e arquivos com responsabilidades claras.	Média	Implementado
RNF08	Deploy em nuvem	A aplicação deve ser publicada em ambiente cloud para acesso externo.	Alta	Implementado
RNF09	Execução do back-end como serviço	O back-end deve permanecer rodando em segundo plano no servidor.	Média	Implementado
RNF10	Servidor web para o front-end	O front-end deve ser servido por um servidor web em produção.	Média	Implementado
RNF11	Senhas criptografadas	As senhas dos usuários devem ser armazenadas de forma criptografada.	Alta	Não implementado
RNF12	Autenticação com token	As rotas administrativas devem ser protegidas por autenticação segura.	Alta	Não implementado
RNF13	HTTPS	A aplicação deve utilizar HTTPS em produção.	Média	Não implementado
RNF14	Banco escalável	O sistema deve utilizar banco mais adequado para produção, como PostgreSQL ou MySQL.	Média	Não implementado

4.3. Backlog do Produto

O backlog foi organizado de acordo com as funcionalidades necessárias para construção do MVP. As tarefas foram classificadas conforme sua implementação no projeto.

ID	Item do Backlog	Descrição	Status
B01	Configurar estrutura do projeto	Separar o projeto em back-end, front-end e documentação.	Done
B02	Criar banco SQLite	Configurar banco de dados local para persistência.	Done
B03	Criar models	Definir entidades de produtos, categorias e usuários.	Done
B04	Criar schemas	Definir estruturas de entrada e saída da API.	Done
B05	Criar CRUD de produtos	Implementar criação, listagem, edição e exclusão de produtos.	Done
B06	Criar API de categorias	Implementar consulta e cadastro de categorias.	Done
B07	Criar CRUD de usuários	Implementar estrutura básica de usuários no back-end.	Done
B08	Configurar CORS	Permitir comunicação entre React e FastAPI.	Done
B09	Criar front-end React/Vite	Implementar interface principal da aplicação.	Done
B10	Consumir API no front-end	Carregar produtos e categorias dinamicamente.	Done
B11	Criar busca por produto	Permitir pesquisa pelo nome do produto.	Done
B12	Criar filtro por categoria	Permitir filtro por categoria.	Done
B13	Implementar carrinho local	Criar carrinho com adição, remoção, quantidade e total.	Done
B14	Persistir carrinho	Salvar carrinho no navegador via <code>localStorage</code> .	Done
B15	Criar login simples de administrador	Implementar acesso administrativo por senha.	Done
B16	Ocultar ações administrativas	Exibir ações administrativas apenas no modo admin.	Done

ID	Item do Backlog	Descrição	Status
B17	Adicionar imagens reais	Substituir imagens genéricas por imagens reais dos produtos.	Done
B18	Fazer deploy na AWS EC2	Publicar a aplicação em ambiente cloud.	Done
B19	Configurar Nginx	Servir o front-end buildado na porta 80.	Done
B20	Configurar systemd	Manter o back-end rodando como serviço.	Done

4.4. Itens Planejados para Evolução

Algumas funcionalidades foram planejadas, mas ficaram fora do escopo final do MVP.

ID	Item	Descrição	Status
B21	Checkout	Criar fluxo de finalização de compra.	Futuro
B22	Pedidos	Persistir pedidos e itens do pedido no banco.	Futuro
B23	Autenticação real	Implementar login com e-mail, senha criptografada e token.	Futuro
B24	Proteção de rotas	Restringir rotas administrativas apenas a administradores autenticados.	Futuro
B25	Histórico de pedidos	Permitir consulta de pedidos anteriores.	Futuro
B26	Painel administrativo completo	Criar painel separado para administração.	Futuro
B27	Migração de banco	Substituir SQLite por PostgreSQL ou MySQL.	Futuro
B28	Domínio e HTTPS	Configurar domínio próprio e certificado SSL.	Futuro

4.5. Priorização das Tarefas

A priorização das tarefas foi definida com base na construção de um MVP funcional. Inicialmente, foram priorizadas as tarefas de estruturação do back-end, banco de dados, models, schemas e rotas da API.

Em seguida, foram desenvolvidas as funcionalidades visíveis ao usuário, como catálogo, busca, filtro por categoria e exibição dos produtos. Após a integração entre front-end e back-end, foram implementadas funcionalidades complementares, como carrinho local, imagens reais e área administrativa simples.

Por fim, foi realizado o deploy em ambiente cloud, utilizando AWS EC2, Nginx para servir o front-end e Uvicorn com systemd para manter o back-end em execução.

4.6. Considerações sobre os Requisitos

A versão final do MVP atendeu aos principais requisitos relacionados ao catálogo de produtos, integração entre front-end e back-end, carrinho local, área administrativa simples e deploy em nuvem.

Alguns requisitos inicialmente planejados, como checkout real, autenticação segura, pedidos e pagamento, não foram implementados nesta etapa. Esses itens foram mantidos como melhorias futuras, pois exigem maior complexidade técnica e regras adicionais de negócio.

Dessa forma, os requisitos implementados foram suficientes para entregar uma aplicação full-stack funcional, demonstrável e alinhada ao objetivo acadêmico do projeto.

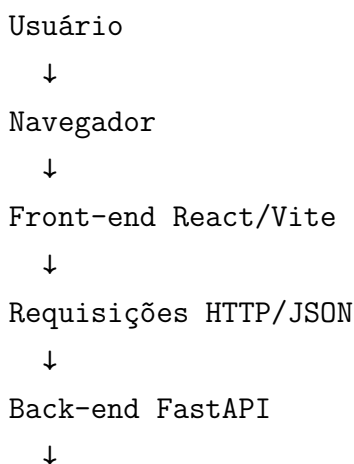
5. Arquitetura da Solução

5.1. Visão Geral da Arquitetura

A solução foi desenvolvida com uma arquitetura full-stack, separando a aplicação em três partes principais: front-end, back-end e banco de dados.

O front-end é responsável pela interface visual e pela interação com o usuário. O back-end é responsável por disponibilizar a API REST, processar as requisições e se comunicar com o banco de dados. O banco de dados é responsável por armazenar as informações utilizadas pelo sistema, como produtos, categorias e usuários.

A arquitetura geral pode ser representada da seguinte forma:



SQLAlchemy
↓
Banco SQLite

Essa estrutura foi escolhida para organizar melhor as responsabilidades do sistema e aproximar o projeto de uma aplicação web real.

5.2. Separação entre Front-end e Back-end

A separação entre front-end e back-end foi uma das principais decisões de arquitetura do projeto.

O front-end foi desenvolvido com **React** e **Vite**, sendo responsável por exibir a interface, controlar os estados da aplicação, consumir a API, apresentar os produtos, gerenciar o carrinho local e controlar a visualização da área administrativa.

O back-end foi desenvolvido com **Python** e **FastAPI**, sendo responsável por disponibilizar os endpoints da API, validar dados, acessar o banco de dados e retornar respostas em formato JSON.

Essa separação permite que cada parte do sistema tenha uma responsabilidade bem definida. Além disso, facilita a manutenção, os testes e futuras evoluções, pois o front-end e o back-end podem ser alterados de forma independente.

5.3. Comunicação via API REST

A comunicação entre front-end e back-end ocorre por meio de uma API REST. O front-end realiza requisições HTTP para buscar, criar, editar ou excluir dados no back-end.

As operações seguem o padrão dos métodos HTTP:

- **GET:** consulta dados.
- **POST:** cria novos registros.
- **PUT:** atualiza registros existentes.
- **DELETE:** remove registros.

No projeto, as principais rotas utilizadas foram:

Produtos:

GET /catalog/

POST /catalog/

PUT /catalog/{product_id}

DELETE /catalog/{product_id}

Categorias:

GET /categories/

POST /categories/

Usuários:

GET /users/

POST /users/

PUT /users/{user_id}

DELETE /users/{user_id}

As respostas da API são retornadas em formato JSON, o que facilita o consumo dos dados pelo React.

5.4. Camadas da Aplicação

A aplicação foi organizada em camadas, cada uma com uma responsabilidade específica.

A **camada de apresentação** corresponde ao front-end React. Ela é responsável pela interface do usuário, incluindo catálogo, cards de produtos, busca, filtro, carrinho, login administrativo e formulários.

A **camada de aplicação** corresponde ao back-end FastAPI. Ela recebe as requisições, executa as operações necessárias e retorna as respostas para o front-end.

A **camada de dados** é composta pelo SQLAlchemy e pelo banco SQLite. Ela é responsável pela persistência das informações utilizadas pela aplicação.

Essa divisão torna o sistema mais organizado e facilita a manutenção do código.

5.5. Arquitetura do Back-end

O back-end foi organizado dentro da pasta `src/app`, separando suas responsabilidades em três camadas principais:

- **db:** configuração e models do banco de dados.

- **schemas:** validação e estrutura dos dados da API.
- **api:** rotas e endpoints do sistema.

O arquivo `main.py` funciona como ponto de entrada da aplicação FastAPI. Ele inicializa a API, configura o CORS e registra os routers responsáveis pelas rotas de produtos, categorias e usuários.

Essa estrutura evita concentrar todo o código em um único arquivo e permite que o sistema seja expandido com mais facilidade.

5.6. Arquitetura do Front-end

O front-end segue a estrutura de um projeto React com Vite.

A pasta `src` contém o código principal da interface, incluindo o componente `App.jsx`, onde estão os estados, chamadas para a API, lógica do carrinho, filtro, busca e área administrativa.

A pasta `public` armazena arquivos públicos, como as imagens reais dos produtos. Essas imagens são acessadas por caminhos como `/products/nome-da-imagem.png`.

O Vite foi utilizado para facilitar o desenvolvimento e gerar o build final da aplicação. Em desenvolvimento, o sistema roda com `npm run dev`. Para produção, é gerada a pasta `dist` por meio do comando `npm run build`.

5.7. Arquitetura de Deploy

O deploy foi realizado em uma instância **EC2 Ubuntu** no **AWS Academy Learner Lab**.

A aplicação foi publicada da seguinte forma:

AWS EC2 Ubuntu

```
|-- Nginx
|   '-- Serve o front-end React na porta 80
|
|-- FastAPI + Uvicorn
|   '-- Executa o back-end na porta 8000
|
|-- SQLite
|   '-- Armazena produtos, categorias e usuários
|
```

```
'-- systemd
    '-- Mantém o back-end rodando como serviço
```

O front-end é acessado pela porta 80, enquanto o back-end é acessado pela porta 8000. Para permitir o acesso externo, o Security Group da AWS foi configurado com as portas necessárias.

As principais portas utilizadas foram:

- **22:** acesso SSH à instância.
- **80:** acesso ao front-end.
- **8000:** acesso à API FastAPI.

5.8. CORS

Como o front-end e o back-end rodam em portas diferentes, foi necessário configurar o CORS no FastAPI.

O CORS permite que o navegador autorize requisições entre origens diferentes. No projeto, essa configuração foi necessária porque o front-end é acessado por uma porta e a API por outra.

Para o MVP acadêmico, foi utilizada uma configuração permissiva, permitindo a comunicação entre o front-end publicado e a API. Em uma versão de produção, o ideal seria restringir as origens permitidas apenas ao domínio oficial da aplicação.

5.9. Justificativa da Arquitetura Escolhida

A arquitetura escolhida foi adequada ao objetivo do projeto por equilibrar simplicidade, organização e funcionalidade.

O uso de **FastAPI** permitiu criar uma API REST de forma clara e objetiva. O uso de **React com Vite** possibilitou criar uma interface dinâmica e de fácil manutenção. O **SQLite** foi suficiente para o escopo acadêmico do MVP, e a **EC2** permitiu publicar a aplicação em ambiente cloud.

A separação entre front-end e back-end tornou o sistema mais modular e preparado para futuras evoluções, como autenticação real, checkout, registro de pedidos e migração para um banco de dados mais robusto.

6. Estrutura do Back-end

6.1. Visão Geral

O back-end do projeto foi desenvolvido em **Python**, utilizando o framework **FastAPI**. Sua principal função é disponibilizar uma API REST para que o front-end consiga consultar, cadastrar, editar e excluir informações relacionadas ao catálogo de produtos, categorias e usuários.

O back-end atua como a camada intermediária entre a interface do usuário e o banco de dados. Ele recebe requisições HTTP enviadas pelo front-end, processa essas requisições, acessa o banco quando necessário e retorna respostas em formato JSON.

A estrutura geral do back-end foi organizada da seguinte forma:

```
backend/  
  src/  
    app/  
      api/  
      db/  
      schemas/  
      main.py  
    seed.py  
    requirements.txt
```

Essa organização foi adotada para separar responsabilidades e facilitar a manutenção do projeto.

6.2. Organização em `src/app`

A pasta `src/app` concentra o código principal da aplicação FastAPI. Essa estrutura foi utilizada para separar o código-fonte da aplicação dos arquivos auxiliares do projeto, como scripts, dependências e banco de dados local.

Dentro de `src/app`, o back-end foi dividido em três partes principais:

- **db:** responsável pela configuração e modelagem do banco de dados.
- **schemas:** responsável pela validação e definição do formato dos dados.
- **api:** responsável pelas rotas e endpoints da aplicação.

O arquivo `main.py` funciona como ponto de entrada da API, sendo responsável por inicializar a aplicação, configurar o CORS e registrar os routers utilizados pelo sistema.

6.3. Camada db

A camada `db` contém os arquivos relacionados ao banco de dados.

O arquivo `database.py` configura a conexão com o banco **SQLite** e cria os elementos necessários para que o **SQLAlchemy** consiga se comunicar com o banco.

O arquivo `models.py` define as entidades persistidas no sistema. No projeto, foram criadas as seguintes entidades principais:

- **DBProduct:** representa os produtos do catálogo.
- **DBCategory:** representa as categorias dos produtos.
- **DBUser:** representa os usuários cadastrados no sistema.

O uso do **SQLAlchemy** permitiu representar as tabelas do banco por meio de classes Python, facilitando operações como criação, consulta, atualização e exclusão de registros.

6.4. Camada schemas

A camada `schemas` contém os modelos criados com **Pydantic**. Esses modelos definem o formato dos dados que entram e saem da API.

Enquanto os `models` representam a estrutura do banco de dados, os `schemas` representam a estrutura dos dados utilizados nas requisições e respostas da API.

Foram criados `schemas` para produtos, categorias e usuários. Essa separação melhora a organização do código e permite validar os dados antes que eles sejam persistidos no banco. Também permite controlar quais informações são retornadas ao front-end.

6.5. Camada api

A camada `api` contém as rotas da aplicação. As rotas foram organizadas em arquivos separados dentro da pasta `routers`, de acordo com a responsabilidade de cada recurso.

Os principais routers implementados foram:

- **catalog.py**: responsável pelas rotas de produtos.
- **categories.py**: responsável pelas rotas de categorias.
- **users.py**: responsável pelas rotas de usuários.

Essa separação evita concentrar todas as rotas no arquivo principal da aplicação e torna o projeto mais organizado.

As principais rotas implementadas foram:

Produtos:

```
GET /catalog/  
POST /catalog/  
PUT /catalog/{product_id}  
DELETE /catalog/{product_id}
```

Categorias:

```
GET /categories/  
POST /categories/
```

Usuários:

```
GET /users/  
POST /users/  
PUT /users/{user_id}  
DELETE /users/{user_id}
```

Essas rotas permitem que o front-end consulte os dados do sistema e que o administrador gerencie o catálogo.

6.6. Banco de Dados

O banco de dados utilizado foi o **SQLite**, escolhido por sua simplicidade e facilidade de configuração para um projeto acadêmico.

O SQLite armazena os dados em um arquivo local, eliminando a necessidade de configurar um servidor de banco de dados separado. Para o escopo do MVP, essa escolha foi suficiente para persistir produtos, categorias e usuários.

A relação mais importante do sistema ocorre entre produtos e categorias. Cada produto pode estar associado a uma categoria, permitindo que o front-end realize filtros e organize melhor a exibição do catálogo.

6.7. Seed de Dados

O arquivo `seed.py` foi utilizado para popular o banco de dados com registros iniciais.

Esse script permitiu cadastrar categorias e produtos base de forma automática, facilitando os testes locais e a configuração inicial do sistema no ambiente de nuvem.

Com o `seed.py`, foi possível recriar rapidamente o catálogo sem precisar cadastrar manualmente todos os produtos pela interface administrativa.

6.8. Funcionamento Geral do Back-end

O funcionamento do back-end segue o seguinte fluxo:

1. O front-end envia uma requisição HTTP para a API.
2. O FastAPI recebe a requisição por meio de uma rota.
3. Os dados são validados pelos schemas Pydantic.
4. A rota acessa o banco de dados utilizando SQLAlchemy.
5. O banco SQLite retorna ou atualiza as informações solicitadas.
6. A API devolve uma resposta em JSON para o front-end.
7. O front-end utiliza os dados recebidos para atualizar a interface.

Esse fluxo garante a separação entre interface, lógica de acesso aos dados e persistência.

6.9. Justificativa da Estrutura

A estrutura do back-end foi definida com foco em organização, manutenção e possibilidade de evolução futura.

A separação entre `db`, `schemas` e `api` permite que cada parte do sistema tenha uma responsabilidade clara. Isso facilita a leitura do código, a correção de problemas e a implementação de novas funcionalidades.

Caso o sistema seja expandido futuramente, novas entidades, como pedidos ou pagamentos, podem ser adicionadas seguindo a mesma estrutura, com novos `models`, `schemas` e `routers`.

Dessa forma, o back-end atende ao escopo do MVP desenvolvido e fornece uma base organizada para futuras melhorias, como autenticação real, proteção de rotas administrativas e registro de pedidos no banco de dados.

7. Estrutura do Front-end

7.1. Visão Geral

O front-end do projeto foi desenvolvido com **React** e **Vite**. Ele é responsável pela interface visual da aplicação e pela interação direta com o usuário.

Por meio do front-end, o usuário consegue visualizar o catálogo de produtos, buscar itens por nome, filtrar produtos por categoria, adicionar produtos ao carrinho e acessar a área administrativa simples. O front-end também é responsável por consumir os dados disponibilizados pelo back-end FastAPI.

A estrutura geral do front-end foi organizada da seguinte forma:

```
frontend/  
  public/  
    products/  
  src/  
    App.jsx  
    main.jsx  
    index.css  
  index.html  
  package.json  
  vite.config.js
```

Essa organização segue o padrão de projetos React com Vite, separando código-fonte, arquivos públicos e arquivos de configuração.

7.2. React e Vite

O **React** foi utilizado para construir a interface da aplicação de forma dinâmica. Sua escolha se justifica pela necessidade de controlar diferentes estados da tela, como produtos carregados, categoria selecionada, busca, carrinho e modo administrador.

O **Vite** foi utilizado como ferramenta de desenvolvimento e build do front-end. Ele facilita a execução local do projeto, oferece atualização rápida durante o desenvolvimento e gera uma versão otimizada para publicação.

Durante o desenvolvimento, o front-end pode ser executado com `npm run dev`. Para publicação, é utilizado `npm run build`. Esse comando gera a pasta `dist`, que contém os arquivos finais do site prontos para serem servidos em produção.

7.3. Pasta `src`

A pasta `src` contém o código principal da aplicação React.

Os principais arquivos são:

- **`main.jsx`**: responsável por inicializar o React e renderizar o componente principal da aplicação.
- **`App.jsx`**: responsável pela maior parte da lógica da interface, incluindo estados, consumo da API, carrinho, filtros, busca, área administrativa e renderização dos produtos.
- **`index.css`**: responsável pelos estilos globais e integração com as classes de estilização utilizadas no projeto.

O arquivo `App.jsx` concentra a principal lógica visual da aplicação. Nele estão as funções responsáveis por carregar dados da API, filtrar produtos, gerenciar o carrinho, controlar o login simples de administrador e enviar requisições para criação, edição e exclusão de produtos.

7.4. Pasta `public`

A pasta `public` armazena arquivos públicos utilizados pela aplicação. No projeto, ela foi utilizada principalmente para armazenar as imagens reais dos produtos.

As imagens ficam organizadas em `frontend/public/products/`. No Vite, arquivos dentro de `public` podem ser acessados diretamente pela raiz do site. Por exemplo, uma imagem salva em `frontend/public/products/barra-pente.png` pode ser acessada pela aplicação como `/products/barra-pente.png`.

Essa estratégia permitiu associar produtos cadastrados no banco a imagens reais, utilizando o campo `image_url`.

7.5. Estados no React

O front-end utiliza estados para controlar as informações que mudam durante o uso da aplicação.

Os principais estados utilizados foram:

- **products:** armazena os produtos retornados pela API.
- **categories:** armazena as categorias retornadas pela API.
- **search:** armazena o texto digitado na busca.
- **selectedCategory:** armazena a categoria selecionada no filtro.
- **cartItems:** armazena os itens adicionados ao carrinho.
- **isCartOpen:** controla se o painel do carrinho está aberto.
- **isAdmin:** controla se o usuário está no modo administrador.
- **showAdminLogin:** controla a exibição do modal de login administrativo.
- **showForm:** controla a exibição do formulário de cadastro e edição de produtos.

O uso de estados foi essencial para manter a interface dinâmica. Sempre que um estado é alterado, o React atualiza automaticamente a parte correspondente da tela.

7.6. Consumo da API

O front-end consome os dados do back-end por meio de requisições HTTP utilizando `fetch`.

A aplicação utiliza uma constante para definir o endereço da API: `API_BASE_URL`. Durante o desenvolvimento local, essa constante aponta para o back-end em `http://127.0.0.1:8000`. No ambiente em nuvem, ela foi ajustada para o IP público da instância EC2.

As principais chamadas realizadas pelo front-end são:

```
GET /catalog/
GET /categories/
POST /catalog/
PUT /catalog/{product_id}
DELETE /catalog/{product_id}
```

Essas chamadas permitem carregar os produtos e categorias, além de executar as ações administrativas de cadastro, edição e exclusão de produtos.

7.7. Busca e Filtro

A busca e o filtro foram implementados no próprio front-end a partir dos produtos carregados da API.

A busca utiliza o texto digitado pelo usuário para comparar com o nome dos produtos. O filtro utiliza a categoria selecionada para exibir apenas os produtos pertencentes à categoria correspondente.

Essa abordagem evita requisições adicionais ao back-end a cada interação do usuário, tornando a navegação mais rápida para o escopo do MVP.

7.8. Carrinho Local

O carrinho foi implementado no front-end utilizando estados e `localStorage`.

O usuário pode adicionar produtos ao carrinho, alterar quantidades, remover itens e limpar o carrinho. O total é calculado automaticamente com base no preço e na quantidade dos produtos.

O uso do `localStorage` permite que os itens do carrinho permaneçam salvos mesmo após o recarregamento da página.

Essa solução foi escolhida por ser adequada ao escopo do MVP, simulando o comportamento de um carrinho sem a necessidade de criar pedidos no banco de dados.

7.9. Área Administrativa

A área administrativa foi implementada de forma simples no front-end.

O usuário pode acessar o modo administrador por meio de uma senha definida na aplicação. Após o login, o sistema exibe opções adicionais, como cadastrar, editar e excluir produtos.

Quando o modo administrador não está ativo, essas ações permanecem ocultas para o usuário comum.

Essa abordagem atende ao objetivo do MVP, permitindo separar a experiência do cliente da experiência de administração. Em uma versão futura, essa funcionalidade pode ser substituída por autenticação real no back-end.

7.10. Build e Publicação

Para publicação, o front-end foi convertido em arquivos estáticos por meio do comando `npm run build`. Esse processo gera a pasta `dist`, contendo a versão final otimizada do site.

No deploy em nuvem, o conteúdo da pasta `dist` foi copiado para o diretório do Nginx: `/var/www/html/`. Dessa forma, o front-end passou a ser servido pela porta 80 da instância EC2.

7.11. Justificativa da Estrutura

A estrutura do front-end foi definida para manter o projeto organizado, facilitar a integração com o back-end e permitir uma boa experiência de uso.

O uso de React permitiu criar uma interface dinâmica baseada em estados. O Vite facilitou o desenvolvimento, o build e a publicação da aplicação. A pasta `public` permitiu organizar as imagens reais dos produtos, enquanto a pasta `src` concentrou o código principal da interface.

Dessa forma, o front-end atende ao escopo do MVP e fornece uma base adequada para futuras melhorias, como criação de páginas separadas, painel administrativo completo, checkout e autenticação real.

8. Funcionalidades Implementadas

8.1. Visão Geral

O MVP desenvolvido para o E-Commerce TDV contempla as principais funcionalidades necessárias para a apresentação e gerenciamento de um catálogo de produtos elétricos.

As funcionalidades implementadas permitem que o usuário visualize produtos, filtre categorias, pesquise itens, utilize um carrinho local e acesse uma área administrativa simples. Além disso, o sistema conta com integração entre front-end e back-end, persistência em banco de dados e deploy em ambiente cloud.

8.2. Catálogo de Produtos

O catálogo de produtos é a funcionalidade central da aplicação. Ele permite exibir os produtos cadastrados no banco de dados, apresentando informações importantes para o usuário.

Cada produto é exibido com nome, descrição, preço, estoque, categoria, imagem e status de disponibilidade.

Os dados são carregados dinamicamente a partir da API FastAPI, por meio da rota de produtos. Dessa forma, o front-end não possui produtos fixos no código; ele consome as informações retornadas pelo back-end.

Essa funcionalidade permite que o catálogo seja atualizado sempre que o administrador cadastra, edita ou exclui um produto.

8.3. Categorias

O sistema possui categorias para organizar os produtos do catálogo. As categorias são cadastradas no back-end e retornadas para o front-end por meio da API.

No front-end, essas categorias são utilizadas para montar o filtro de produtos. Isso permite que o usuário visualize apenas os itens pertencentes a uma categoria específica.

Essa funcionalidade melhora a navegação e facilita a localização de produtos dentro do catálogo.

8.4. Busca por Produto

A busca permite que o usuário encontre produtos pelo nome. O campo de busca está disponível na interface principal do catálogo.

Ao digitar um termo, o sistema filtra os produtos carregados e exibe apenas aqueles cujo nome corresponde ao texto informado.

Essa busca foi implementada no front-end, utilizando os dados já carregados da API. Essa abordagem reduz a necessidade de novas requisições ao back-end durante a digitação.

8.5. Filtro por Categoria

O filtro por categoria permite ao usuário visualizar produtos de uma categoria específica.

O front-end compara a categoria selecionada com a categoria associada a cada produto. Quando uma categoria é selecionada, apenas os produtos correspondentes são exibidos.

Também existe a opção de visualizar todas as categorias, permitindo retornar à listagem completa de produtos.

8.6. Carrinho Local

O carrinho local permite que o usuário adicione produtos, remova itens, altere quantidades e visualize o total da compra.

As principais ações do carrinho são adicionar produto, aumentar quantidade, diminuir quantidade, remover item, limpar carrinho e calcular valor total.

O carrinho foi implementado no front-end, utilizando estados do React e persistência no `localStorage`. Dessa forma, os itens adicionados permanecem salvos mesmo após o recarregamento da página.

Essa funcionalidade simula o comportamento de um carrinho de compras sem a necessidade de registrar pedidos no banco de dados.

8.7. Área Administrativa Simples

A aplicação possui uma área administrativa simples, acessada por senha no front-end.

Quando o usuário entra no modo administrador, o sistema exibe ações adicionais que não aparecem para o cliente comum, como novo produto, editar produto e excluir produto.

Essa separação permite diferenciar a experiência do cliente da experiência do administrador. O cliente comum consegue apenas visualizar produtos e utilizar o carrinho, enquanto o administrador consegue gerenciar o catálogo.

No MVP, essa área administrativa foi implementada como controle visual no front-end. Em uma versão futura, essa funcionalidade poderá ser substituída por autenticação real no back-end.

8.8. Cadastro de Produtos

O administrador consegue cadastrar novos produtos por meio de um formulário na interface.

O formulário permite informar nome, descrição, preço, estoque, categoria, URL da imagem e status ativo/inativo.

Após o envio, o front-end realiza uma requisição `POST` para a API. O back-end valida os dados, salva o produto no banco SQLite e retorna o produto cadastrado.

Depois do cadastro, a lista de produtos é atualizada no front-end.

8.9. Edição de Produtos

A edição de produtos permite que o administrador altere informações de itens já cadastrados.

Ao clicar em “Editar”, o formulário é preenchido com os dados atuais do produto. O administrador pode alterar as informações e salvar as mudanças.

O front-end envia uma requisição `PUT` para o back-end, informando o identificador do produto e os novos dados. Após a atualização, o catálogo é recarregado para refletir as alterações.

8.10. Exclusão de Produtos

A exclusão de produtos permite que o administrador remova itens do catálogo.

Antes da exclusão, o sistema solicita uma confirmação para evitar remoções acidentais. Após a confirmação, o front-end envia uma requisição `DELETE` para a API.

O back-end remove o produto do banco de dados, e o front-end atualiza a listagem exibida.

8.11. Imagens Reais dos Produtos

Uma melhoria importante implementada no projeto foi a substituição de imagens genéricas por imagens reais dos produtos da TDV Elétrica.

As imagens foram armazenadas na pasta pública do front-end: `frontend/public/products/`.

No banco de dados, cada produto armazena o caminho da imagem no campo `image_url`, seguindo o padrão `/products/nome-da-imagem.png`.

Essa estratégia permite que o front-end exiba imagens reais nos cards dos produtos, melhorando a apresentação visual do catálogo.

8.12. Integração com API

O front-end foi integrado ao back-end por meio de requisições HTTP.

As principais integrações realizadas foram buscar produtos, buscar categorias, cadastrar produto, editar produto e excluir produto.

Essa comunicação ocorre por meio da API REST desenvolvida com FastAPI. As respostas são retornadas em JSON e utilizadas pelo React para atualizar a interface.

8.13. Deploy da Aplicação

A aplicação foi publicada em ambiente cloud utilizando uma instância EC2 no AWS Academy Learner Lab.

O back-end foi executado com Uvicorn na porta 8000 e configurado como serviço com `systemd`. O front-end foi buildado com Vite e servido pelo Nginx na porta 80.

Com isso, o sistema passou a ser acessível por meio de um IP público, permitindo testes fora do ambiente local de desenvolvimento.

8.14. Considerações sobre as Funcionalidades

As funcionalidades implementadas atendem ao escopo principal do MVP, permitindo demonstrar o funcionamento de uma aplicação full-stack integrada.

O sistema entrega um catálogo funcional, com produtos reais, carrinho local, área administrativa simples, persistência em banco de dados e deploy em nuvem.

Funcionalidades mais avançadas, como checkout, pedidos, autenticação segura e pagamento, foram mantidas como melhorias futuras.

9. Modelagem do Sistema

9.1. Visão Geral da Modelagem

A modelagem do sistema foi utilizada para representar, de forma visual e conceitual, os principais elementos da aplicação e suas interações. Ela serviu como apoio para compreender os atores envolvidos, as funcionalidades esperadas, as entidades principais e a arquitetura geral da solução.

A documentação inicial do projeto contemplava diagramas de casos de uso, dia-

grama de domínio de classes, diagramas de sequência, diagrama de arquitetura e diagrama de implantação. Na documentação final, esses artefatos foram mantidos como referência do planejamento e atualizados conceitualmente conforme o escopo implementado no MVP.

A modelagem final considera principalmente as funcionalidades entregues: catálogo de produtos, categorias, carrinho local, área administrativa simples, CRUD de produtos, estrutura de usuários e deploy em nuvem.

9.2. Casos de Uso

Os casos de uso representam as principais interações entre os usuários e o sistema. No projeto, foram considerados dois atores principais:

- **Cliente:** usuário que acessa o site para visualizar produtos, pesquisar, filtrar e utilizar o carrinho.
- **Administrador:** usuário responsável por gerenciar o catálogo de produtos.

Os principais casos de uso do MVP são:

- UC01 — Pesquisar Produto.
- UC02 — Visualizar Produtos.
- UC03 — Filtrar Produtos por Categoria.
- UC04 — Organizar Carrinho de Compras.
- UC05 — Acessar Área Administrativa.
- UC06 — Cadastrar Produto.
- UC07 — Editar Produto.
- UC08 — Excluir Produto.
- UC09 — Consultar Categorias.

Funcionalidades inicialmente planejadas, como checkout, pagamento, rastreamento de pedido e cupom de desconto, foram mantidas como possibilidades de evolução futura, mas não fazem parte do escopo implementado no MVP.

9.3. Descrição dos Principais Casos de Uso

UC01 — Pesquisar Produto: o cliente informa um termo de busca no campo de pesquisa. O sistema filtra os produtos carregados e exibe apenas os itens cujo nome corresponde ao termo digitado.

UC02 — Visualizar Produtos: o cliente acessa a página principal da aplicação. O sistema consulta a API, recupera os produtos cadastrados e exibe as informações em cards.

UC03 — Filtrar Produtos por Categoria: o cliente seleciona uma categoria no filtro. O sistema exibe apenas os produtos associados à categoria selecionada.

UC04 — Organizar Carrinho de Compras: o cliente adiciona produtos ao carrinho, altera quantidades, remove itens ou limpa o carrinho. O sistema atualiza o total automaticamente e mantém os dados salvos no navegador por meio do `localStorage`.

UC05 — Acessar Área Administrativa: o administrador acessa a área administrativa por meio de uma senha simples no front-end. Após a validação, o sistema exibe as opções de gerenciamento do catálogo.

UC06 — Cadastrar Produto: o administrador preenche o formulário com os dados de um novo produto. O sistema envia as informações para a API, que valida os dados e salva o produto no banco.

UC07 — Editar Produto: o administrador seleciona um produto existente e altera suas informações. O sistema envia os dados atualizados para a API, que persiste as mudanças no banco.

UC08 — Excluir Produto: o administrador solicita a exclusão de um produto. Após confirmação, o sistema envia a requisição para a API, que remove o produto do banco.

UC09 — Consultar Categorias: o sistema consulta as categorias cadastradas no back-end e utiliza essas informações para montar o filtro de categorias no front-end.

9.4. Diagrama de Domínio

O diagrama de domínio representa as principais entidades do sistema e seus relacionamentos.

No MVP implementado, as entidades principais são: Produto, Categoria, Usuário, Carrinho e Item do Carrinho.

A entidade **Produto** representa os itens comercializados pela TDV Elétrica. Cada produto possui nome, descrição, preço, estoque, imagem e status de atividade.

A entidade **Categoria** organiza os produtos em grupos, permitindo que o usuário filtre os itens exibidos no catálogo.

A entidade **Usuário** representa a estrutura inicial para cadastro de usuários no back-end. Embora a autenticação real não tenha sido implementada no MVP, essa entidade prepara o sistema para futuras evoluções.

O **Carrinho** e o **Item do Carrinho** são representados no front-end, pois o carrinho foi implementado localmente no navegador. Essa estrutura permite adicionar produtos, alterar quantidades e calcular o total.

O principal relacionamento persistido no banco ocorre entre produto e categoria. Um produto pode estar associado a uma categoria, e uma categoria pode agrupar vários produtos.

9.5. Diagramas de Sequência

Os diagramas de sequência representam o fluxo de comunicação entre usuário, front-end, back-end e banco de dados.

No projeto final, os fluxos mais relevantes são carregamento do catálogo, adição de produto ao carrinho, cadastro de produto pelo administrador, edição de produto pelo administrador e exclusão de produto pelo administrador.

No fluxo de carregamento do catálogo, o usuário acessa o site, o front-end realiza uma requisição para a API, o back-end consulta o banco de dados e retorna a lista de produtos em JSON. O React utiliza esses dados para renderizar os cards na interface.

No fluxo de cadastro de produto, o administrador preenche o formulário, o front-end envia uma requisição **POST** para a API, o back-end valida os dados, salva o registro no banco e retorna o produto cadastrado.

Os fluxos de edição e exclusão seguem lógica semelhante, utilizando respectivamente os métodos **PUT** e **DELETE**.

9.6. Diagrama de Arquitetura

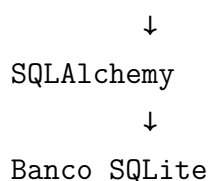
O diagrama de arquitetura representa a divisão do sistema em camadas.

A arquitetura final do projeto pode ser entendida da seguinte forma:

Front-end React/Vite

↓

API REST FastAPI



Essa estrutura separa a interface, a lógica da API e a persistência dos dados. O front-end não acessa o banco diretamente; toda comunicação ocorre por meio da API REST.

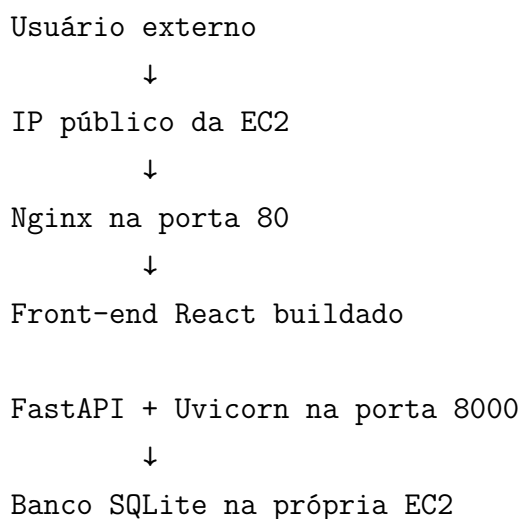
Essa divisão melhora a organização da aplicação e facilita futuras evoluções, como substituição do banco SQLite por PostgreSQL ou criação de novos módulos no back-end.

9.7. Diagrama de Implantação

O diagrama de implantação representa como a aplicação foi publicada em ambiente de nuvem.

No projeto final, a implantação foi realizada em uma instância EC2 Ubuntu no AWS Academy Learner Lab.

A estrutura de implantação é composta por:



O Nginx é responsável por servir o front-end. O back-end é executado com Uvicorn e mantido como serviço com `systemd`. O banco SQLite fica armazenado localmente na instância EC2.

As portas liberadas no Security Group foram 22 para acesso SSH, 80 para acesso ao front-end e 8000 para acesso à API.

Essa configuração permitiu que a aplicação fosse acessada por meio de um IP público, simulando um ambiente de produção.

9.8. Considerações sobre a Modelagem

A modelagem do sistema foi importante para orientar o desenvolvimento e documentar a estrutura da aplicação. Embora algumas funcionalidades planejadas inicialmente tenham ficado fora do MVP, os diagramas e casos de uso ajudaram a definir uma base clara para a implementação.

A versão final do sistema manteve o foco nas funcionalidades essenciais: catálogo, categorias, carrinho local, área administrativa, CRUD de produtos e deploy em nuvem. Essa modelagem também fornece uma base para futuras evoluções, como autenticação real, pedidos, checkout e controle avançado de estoque.

10. Deploy em Nuvem

10.1. Visão Geral

O deploy da aplicação foi realizado em ambiente de nuvem utilizando o **AWS Academy Learner Lab**. A estratégia adotada foi hospedar o front-end e o back-end em uma única instância **EC2 Ubuntu**, permitindo que o sistema pudesse ser acessado por meio de um IP público.

Essa etapa foi importante para validar o funcionamento da aplicação fora do ambiente local, simulando uma publicação real. Com o deploy, foi possível acessar o catálogo, consumir a API, visualizar os produtos, carregar imagens reais, utilizar o carrinho e acessar a área administrativa por meio da internet.

10.2. AWS Academy Learner Lab

O AWS Academy Learner Lab foi utilizado como ambiente cloud para hospedagem do projeto. Esse ambiente permite criar recursos da AWS para fins acadêmicos, como instâncias EC2, regras de rede e serviços básicos de infraestrutura.

Devido às características do Learner Lab, foi adotada uma abordagem simples e controlada, utilizando uma instância EC2 para executar toda a aplicação.

A estrutura final ficou organizada da seguinte forma:

EC2 Ubuntu

- |-- Front-end React servido pelo Nginx
- |-- Back-end FastAPI executado com Uvicorn
- |-- Banco SQLite armazenado localmente

```
'-- Serviço systemd para manter o back-end ativo
```

10.3. Instância EC2

A instância EC2 foi utilizada como servidor principal da aplicação. Nela foram instaladas as dependências necessárias para executar o projeto, como Python, Node.js, npm, Git e Nginx.

O repositório do projeto foi clonado a partir do GitHub diretamente na instância, permitindo executar o back-end e gerar o build do front-end no próprio ambiente cloud.

A EC2 funcionou como a máquina responsável por hospedar API FastAPI, front-end React buildado, banco SQLite, arquivos públicos das imagens e servidor Nginx.

10.4. Security Group

O Security Group foi configurado para permitir o acesso externo às portas necessárias da aplicação.

Porta	Serviço	Finalidade
22	SSH	Acesso remoto à instância EC2.
80	HTTP	Acesso ao front-end servido pelo Nginx.
8000	FastAPI	Acesso à API do back-end.

A porta 22 foi utilizada para administração da instância via terminal. A porta 80 foi utilizada para acesso ao site pelo navegador. A porta 8000 foi utilizada para acesso à API FastAPI.

Sem essa configuração, a aplicação poderia estar em execução dentro da EC2, mas não seria acessível externamente.

10.5. Deploy do Back-end

O back-end foi executado com **Uvicorn**, servidor ASGI utilizado para rodar aplicações FastAPI.

O comando base utilizado para execução da API foi:

```
uvicorn src.app.main:app --host 0.0.0.0 --port 8000
```

O parâmetro `-host 0.0.0.0` foi necessário para permitir que a API aceitasse conexões externas. A porta 8000 foi utilizada para disponibilizar os endpoints do back-end.

Após os testes iniciais, o back-end foi configurado como serviço do sistema utilizando **systemd**. Essa configuração permitiu manter a API em execução em segundo plano, mesmo após o fechamento do terminal.

O serviço criado recebeu o nome **tdv-backend**. Com isso, o back-end passou a ser controlado por comandos como `sudo systemctl start tdv-backend`, `sudo systemctl restart tdv-backend` e `sudo systemctl status tdv-backend`.

10.6. Deploy do Front-end

O front-end foi desenvolvido com React e Vite. Para publicação em produção, foi gerado o build da aplicação com o comando `npm run build`.

Esse comando cria a pasta **dist**, contendo os arquivos finais otimizados do site.

Após a geração do build, o conteúdo da pasta **dist** foi copiado para o diretório padrão do Nginx: `/var/www/html/`.

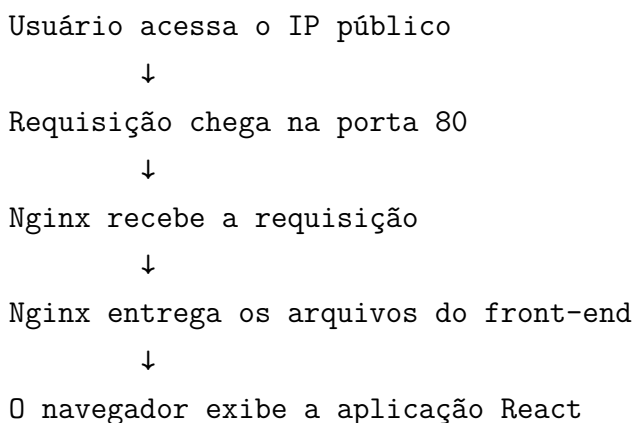
O Nginx passou então a servir a aplicação pela porta 80. Dessa forma, o usuário consegue acessar o front-end diretamente pelo IP público da EC2.

10.7. Nginx

O Nginx foi utilizado como servidor web para disponibilizar o front-end React.

Sua função no projeto foi servir os arquivos estáticos gerados pelo Vite após o build da aplicação.

O fluxo de acesso ao front-end ocorre da seguinte forma:



Essa abordagem é adequada porque, em produção, o front-end React não precisa

rodar com `npm run dev`. Ele deve ser buildado e servido como arquivos estáticos.

10.8. Comunicação entre Front-end e Back-end

Após o deploy, o front-end passou a consumir a API utilizando o IP público da EC2.

A constante `API_BASE_URL` foi ajustada para apontar para o endereço do back-end publicado: `http://IP_PUBLICO_DA_EC2:8000`.

Com isso, o front-end consegue realizar chamadas para endpoints como `/catalog/` e `/categories/`.

Essa configuração permitiu que a aplicação publicada carregasse os produtos e categorias diretamente da API hospedada na mesma instância EC2.

10.9. CORS

Como o front-end e o back-end são acessados em portas diferentes, foi necessário configurar o CORS no FastAPI.

O front-end é acessado pela porta 80, enquanto o back-end é acessado pela porta 8000. Mesmo utilizando o mesmo IP, o navegador considera essas origens diferentes.

A configuração de CORS permitiu que o front-end realizasse requisições para a API sem bloqueio do navegador.

Para o contexto acadêmico do MVP, foi utilizada uma configuração permissiva. Em uma versão de produção, o ideal seria restringir as origens permitidas apenas ao domínio oficial da aplicação.

10.10. Imagens no Ambiente Cloud

As imagens reais dos produtos foram armazenadas no diretório público do front-end: `frontend/public/products/`.

Durante o processo de build, essas imagens foram copiadas para a versão final da aplicação. Após a publicação pelo Nginx, passaram a ser acessíveis por URLs públicas, seguindo o padrão `/products/nome-da-imagem.png`.

Essa estratégia permitiu que os produtos cadastrados no banco mantivessem apenas o caminho relativo da imagem, sem depender de caminhos locais do computador do desenvolvedor.

10.11. Limitações do Deploy

Por utilizar o AWS Academy Learner Lab, algumas limitações devem ser consideradas.

A principal limitação é que o IP público da instância pode mudar quando o laboratório é reiniciado ou quando a instância é parada e iniciada novamente. Quando isso ocorre, é necessário atualizar o endereço da API no front-end e gerar novamente o build da aplicação.

Outra limitação é o uso do SQLite dentro da própria EC2. Para um ambiente de produção real, o ideal seria utilizar um banco de dados externo e gerenciado, como PostgreSQL ou MySQL em um serviço próprio de banco de dados.

Além disso, a aplicação foi publicada utilizando HTTP, sem configuração de domínio próprio ou HTTPS. Esses pontos foram considerados melhorias futuras.

10.12. Considerações sobre o Deploy

O deploy em nuvem permitiu validar o funcionamento da aplicação em um ambiente externo ao computador local. A solução publicada demonstrou a integração completa entre front-end, back-end e banco de dados.

A utilização da EC2 possibilitou compreender etapas importantes de infraestrutura, como configuração de servidor, instalação de dependências, abertura de portas, execução de serviços, build do front-end e publicação com Nginx.

Dessa forma, o deploy atendeu ao objetivo acadêmico do projeto e permitiu disponibilizar uma versão funcional do E-Commerce TDV em ambiente cloud.

11. Testes e Validações

11.1. Visão Geral

Após a implementação das principais funcionalidades do sistema, foram realizados testes para verificar o funcionamento da aplicação em ambiente local e em ambiente cloud. Os testes tiveram como objetivo validar a integração entre front-end, back-end, banco de dados, carrinho local, área administrativa, imagens reais e deploy em nuvem.

Os testes foram realizados de forma manual, utilizando navegador, documentação automática da API FastAPI, terminal da instância EC2 e interação direta com a interface da aplicação.

11.2. Testes do Back-end

Os testes do back-end foram realizados por meio da documentação automática gerada pelo FastAPI, disponível na rota `/docs`, e por acessos diretos aos endpoints da API.

Teste	Resultado Esperado	Status
Acessar <code>/docs</code>	A documentação da API deve abrir corretamente.	Aprovado
Acessar GET <code>/catalog/</code>	A API deve retornar a lista de produtos em JSON.	Aprovado
Acessar GET <code>/categories/</code>	A API deve retornar a lista de categorias em JSON.	Aprovado
Criar produto	A API deve cadastrar um novo produto no banco.	Aprovado
Editar produto	A API deve atualizar os dados do produto.	Aprovado
Excluir produto	A API deve remover o produto selecionado.	Aprovado
Rodar <code>seed.py</code>	O banco deve ser populado com dados iniciais.	Aprovado

Esses testes confirmaram que o back-end estava funcionando corretamente e que os dados estavam sendo persistidos no banco SQLite.

11.3. Testes do Front-end

Os testes do front-end foram realizados diretamente pela interface da aplicação React.

Teste	Resultado Esperado	Status
Abrir página inicial	A página deve carregar sem erro.	Aprovado
Carregar produtos	Os produtos devem aparecer no catálogo.	Aprovado
Exibir imagens	As imagens reais devem aparecer nos cards.	Aprovado
Buscar produto	A busca deve filtrar produtos pelo nome.	Aprovado
Filtrar categoria	O filtro deve exibir produtos da categoria selecionada.	Aprovado
Abrir carrinho	O painel do carrinho deve ser exibido.	Aprovado
Adicionar item ao carrinho	O produto deve ser incluído no carrinho.	Aprovado
Alterar quantidade	A quantidade e o total devem ser atualizados.	Aprovado
Remover item	O produto deve ser removido do carrinho.	Aprovado
Recarregar página	O carrinho deve permanecer salvo pelo <code>localStorage</code> .	Aprovado

Esses testes validaram as principais interações do usuário com a interface.

11.4. Testes da Área Administrativa

A área administrativa foi testada para verificar se as ações de gerenciamento ficavam disponíveis apenas no modo administrador.

Teste	Resultado Esperado	Status
Acessar site como cliente comum	Botões de cadastro, edição e exclusão não devem aparecer.	Aprovado
Abrir modal de admin	O sistema deve solicitar senha.	Aprovado
Inserir senha correta	O modo administrador deve ser ativado.	Aprovado
Inserir senha incorreta	O sistema deve exibir mensagem de erro.	Aprovado
Cadastrar produto	O novo produto deve aparecer no catálogo.	Aprovado
Editar produto	As alterações devem ser refletidas no catálogo.	Aprovado
Excluir produto	O produto deve ser removido da listagem.	Aprovado
Sair do admin	As ações administrativas devem ser ocultadas novamente.	Aprovado

Esses testes confirmaram a separação visual entre cliente comum e administrador.

11.5. Testes de Integração

Os testes de integração tiveram como objetivo verificar a comunicação entre o front-end React e o back-end FastAPI.

Fluxo	Resultado Esperado	Status
Front-end buscar produtos na API	Produtos devem ser carregados dinamicamente.	Aprovado
Front-end buscar categorias na API	Categorias devem aparecer no filtro.	Aprovado
Admin cadastrar produto pelo front-end	Produto deve ser salvo no banco via API.	Aprovado
Admin editar produto pelo front-end	Produto deve ser atualizado no banco.	Aprovado
Admin excluir produto pelo front-end	Produto deve ser removido do banco.	Aprovado

Esses testes confirmaram que a comunicação por API REST estava funcionando conforme esperado.

11.6. Testes de Deploy

Após a publicação em ambiente cloud, foram realizados testes para validar o funcionamento da aplicação na instância EC2.

Teste	Resultado Esperado	Status
Acessar IP público na porta 80	O front-end deve abrir no navegador.	Aprovado
Acessar IP público na porta 8000	A API FastAPI deve responder.	Aprovado
Verificar Nginx	O servidor web deve estar ativo.	Aprovado
Verificar serviço do back-end	O serviço <code>tdv-backend</code> deve estar ativo.	Aprovado
Carregar dados em produção	O front-end publicado deve consumir a API da EC2.	Aprovado
Carregar imagens em produção	As imagens devem ser servidas pelo Nginx.	Aprovado

Esses testes demonstraram que a aplicação estava acessível fora do ambiente local e que o deploy foi realizado com sucesso.

11.7. Validação do Carrinho Local

O carrinho local foi validado para garantir que funcionasse corretamente sem depender do banco de dados.

Foram testadas as ações de adicionar produto, alterar quantidade, remover item, limpar carrinho e recarregar a página. Após o recarregamento, os itens permaneceram salvos no navegador, confirmando o funcionamento do `localStorage`.

Essa validação confirmou que o carrinho atende ao escopo do MVP, simulando uma experiência de compra básica.

11.8. Validação das Imagens dos Produtos

As imagens reais dos produtos foram testadas em ambiente local e em ambiente cloud.

Foi verificado se as imagens estavam salvas na pasta `frontend/public/products/`, se eram copiadas para a pasta `dist` durante o build, se eram publicadas corretamente no Nginx, se o campo `image_url` apontava para o caminho correto e se os cards do catálogo exibiam as imagens reais.

Após os ajustes, as imagens foram carregadas corretamente na aplicação publicada.

11.9. Problemas Encontrados e Soluções

Durante os testes, alguns problemas foram identificados e corrigidos.

Problema	Causa	Solução
Front-end não carregava dados	API apontava para IP antigo ou CORS bloqueava requisição.	Atualização do <code>API_BASE_URL</code> e configuração do CORS.
API não abria externamente	Porta 8000 não estava acessível ou back-end não estava rodando.	Liberação da porta no Security Group e configuração com <code>systemd</code> .

Problema	Causa	Solução
Imagens não apareciam na AWS	Imagens não estavam copiadas para o Nginx ou <code>image_url</code> estava incorreto.	Novo build do front-end e correção dos caminhos das imagens.
IP público mudou	Reinício do ambiente AWS Academy.	Atualização do IP no front-end e novo build.
Vite apresentou erro de permissão	Dependências do Node na EC2 com permissão incorreta.	Reinstalação do <code>node_modules</code> e novo <code>npm install</code> .

Esses problemas foram importantes para validar o comportamento da aplicação em ambiente real de execução.

11.10. Considerações sobre os Testes

Os testes realizados demonstraram que o MVP atende aos principais requisitos definidos para o projeto. A aplicação permite visualizar produtos, carregar categorias, utilizar busca e filtro, gerenciar o carrinho, acessar a área administrativa, modificar o catálogo e utilizar o sistema em ambiente cloud.

Embora os testes tenham sido manuais, eles foram suficientes para validar as funcionalidades principais do projeto acadêmico. Em uma versão futura, poderiam ser adicionados testes automatizados para back-end, front-end e integração.

12. Limitações do Protótipo

12.1. Visão Geral

O sistema desenvolvido atende ao objetivo principal do MVP, entregando uma aplicação full-stack funcional para catálogo e gerenciamento básico de produtos da TDV Elétrica. No entanto, por se tratar de um projeto acadêmico e de uma primeira versão da solução, algumas funcionalidades foram implementadas de forma simplificada ou ficaram fora do escopo final.

As limitações descritas neste capítulo são importantes para demonstrar os pontos que ainda podem ser evoluídos em versões futuras do sistema.

12.2. Login Administrativo Simplificado

A área administrativa foi implementada com um login simples no front-end, utilizando uma senha definida diretamente na aplicação. Essa abordagem foi suficiente para separar visualmente as ações do cliente comum e do administrador durante o MVP.

No entanto, essa solução não representa uma autenticação segura de produção. Como a validação ocorre no front-end, usuários com conhecimento técnico poderiam inspecionar o código da aplicação.

Em uma versão futura, o ideal seria implementar autenticação real no back-end, com usuários cadastrados, senha criptografada e uso de token JWT.

12.3. Rotas Administrativas sem Proteção Real

Embora os botões de cadastro, edição e exclusão fiquem ocultos para usuários comuns no front-end, as rotas administrativas do back-end ainda não possuem proteção por autenticação.

Isso significa que a separação entre cliente e administrador funciona na interface, mas ainda não existe uma camada real de autorização no back-end.

Em uma versão futura, as rotas POST, PUT e DELETE deveriam ser protegidas, permitindo acesso apenas a usuários administradores autenticados.

12.4. Carrinho Local sem Pedido no Banco

O carrinho foi implementado localmente no navegador, utilizando `localStorage`. Essa abordagem permite adicionar produtos, alterar quantidades, remover itens e manter o carrinho salvo após recarregar a página.

No entanto, o carrinho ainda não gera um pedido no banco de dados. Ele funciona como uma simulação da experiência de compra, mas não registra transações, histórico ou status de pedido.

Em uma versão futura, o sistema poderia criar tabelas de pedidos e itens do pedido, permitindo registrar compras reais.

12.5. Ausência de Checkout

O sistema não possui fluxo completo de checkout. A aplicação permite adicionar produtos ao carrinho, mas não finaliza a compra, não solicita dados de entrega, não calcula frete e não gera número de pedido.

Essa funcionalidade foi prevista no planejamento inicial, mas ficou fora do escopo final do MVP devido à maior complexidade técnica.

12.6. Ausência de Pagamento Online

O projeto não possui integração com gateway de pagamento. Portanto, não há processamento de cartão, Pix, boleto ou qualquer outra forma de pagamento.

Essa integração exigiria regras adicionais de segurança, tratamento de dados sensíveis e uso de serviços externos, sendo considerada uma melhoria futura.

12.7. Banco SQLite em Ambiente Cloud

O banco de dados utilizado foi o SQLite, armazenado localmente na própria instância EC2.

Essa escolha foi adequada para o escopo acadêmico, pois simplificou a configuração e permitiu validar a aplicação sem necessidade de um servidor de banco separado.

No entanto, para produção, o SQLite não é a melhor opção em um ambiente cloud escalável. O ideal seria utilizar um banco mais robusto, como PostgreSQL ou MySQL, preferencialmente em um serviço gerenciado, como Amazon RDS.

12.8. IP Público Variável no AWS Academy

Como o deploy foi realizado no AWS Academy Learner Lab, o IP público da instância EC2 pode mudar quando o laboratório é reiniciado ou quando a instância é parada e iniciada novamente.

Quando isso ocorre, é necessário atualizar o endereço da API no front-end e gerar um novo build da aplicação.

Em um ambiente de produção real, o ideal seria utilizar um IP fixo, domínio próprio e HTTPS.

12.9. Ausência de HTTPS

A aplicação foi publicada utilizando HTTP, sem certificado SSL.

Isso significa que o navegador pode indicar que o site não é seguro. Para o escopo acadêmico, isso foi aceitável, mas em um ambiente real seria necessário configurar HTTPS para proteger a comunicação entre usuário e aplicação.

12.10. Responsividade Limitada

A interface foi construída com foco em desktop e notebook. Embora utilize estrutura visual adaptável, ainda poderiam ser feitos ajustes específicos para melhorar a experiência em dispositivos móveis.

Uma versão futura poderia incluir melhorias de responsividade para celulares e tablets.

12.11. Considerações sobre as Limitações

As limitações identificadas não impedem o funcionamento do MVP, mas indicam pontos importantes para evolução futura.

O projeto final cumpre o objetivo acadêmico de demonstrar uma aplicação full-stack integrada, com front-end, back-end, banco de dados, carrinho local, área administrativa simples, imagens reais e deploy em nuvem.

As limitações registradas servem como base para planejar as próximas versões do sistema, aproximando-o gradualmente de uma aplicação de e-commerce real em produção.

13. Melhorias Futuras

13.1. Visão Geral

Embora o MVP desenvolvido atenda aos principais objetivos do projeto, existem diversas melhorias que podem ser implementadas em versões futuras para aproximar o sistema de um e-commerce completo de produção.

As melhorias futuras estão relacionadas principalmente à segurança, autenticação, pedidos, checkout, banco de dados, infraestrutura e experiência do usuário.

13.2. Autenticação Real

Uma das principais melhorias futuras é substituir o login simples de administrador por um sistema real de autenticação.

A versão atual utiliza uma senha definida no front-end, suficiente para o protótipo acadêmico, mas inadequada para produção.

A melhoria proposta inclui login com e-mail e senha, validação de credenciais no back-end, criptografia de senhas, geração de token JWT, controle de sessão do usuário e

diferenciação entre usuário comum e administrador.

Com isso, o acesso administrativo deixaria de ser apenas visual e passaria a ser validado de forma segura pelo back-end.

13.3. Proteção de Rotas Administrativas

Atualmente, as ações administrativas ficam ocultas no front-end para usuários comuns, mas as rotas do back-end ainda não possuem autenticação real.

Em uma versão futura, as rotas de criação, edição e exclusão de produtos devem ser protegidas. As rotas `POST /catalog/`, `PUT /catalog/{product_id}` e `DELETE /catalog/{product_id}` deveriam permitir acesso apenas a usuários autenticados com perfil de administrador.

13.4. Checkout

Outra melhoria importante é a implementação do fluxo de checkout.

O sistema atual possui carrinho local, mas ainda não permite finalizar a compra. Em uma versão futura, o checkout poderia incluir revisão dos itens do carrinho, dados do cliente, endereço de entrega, forma de pagamento, resumo do pedido, confirmação da compra e geração de número do pedido.

Essa funcionalidade permitiria transformar o carrinho em um fluxo de compra mais completo.

13.5. Registro de Pedidos

Com a implementação do checkout, também seria necessário criar a persistência de pedidos no banco de dados.

Para isso, poderiam ser criadas novas entidades, como `Pedido`, `ItemPedido` e `StatusPedido`.

Essas entidades permitiriam registrar cliente responsável pelo pedido, produtos comprados, quantidade de cada item, valor total, status do pedido e data da compra.

Essa evolução permitiria criar histórico de compras e controle de vendas.

13.6. Atualização Automática de Estoque

Atualmente, o estoque é exibido no catálogo e pode ser alterado pelo administrador, mas não é reduzido automaticamente após uma compra.

Em uma versão futura, ao finalizar um pedido, o sistema poderia diminuir automaticamente a quantidade disponível em estoque.

Essa melhoria aproximaria o sistema de uma operação real de e-commerce, evitando que produtos indisponíveis continuem sendo exibidos como disponíveis.

13.7. Pagamento Online

O sistema também poderia evoluir para integrar formas de pagamento online, como Pix, cartão de crédito, boleto e gateway de pagamento externo.

Como pagamentos envolvem dados sensíveis, essa melhoria exigiria atenção especial a segurança, validações e conformidade com boas práticas.

13.8. Painel Administrativo Completo

A área administrativa atual funciona dentro da mesma interface do catálogo. Em uma versão futura, poderia ser criado um painel administrativo separado.

Esse painel poderia incluir dashboard de produtos, gerenciamento de estoque, listagem de pedidos, controle de usuários, cadastro de categorias e relatórios simples de vendas.

Essa separação melhoraria a organização do sistema e tornaria a experiência do administrador mais completa.

13.9. Migração do Banco de Dados

O SQLite foi adequado para o MVP, mas uma versão de produção poderia utilizar um banco de dados mais robusto, como PostgreSQL, MySQL ou Amazon RDS.

Essa migração permitiria maior segurança, escalabilidade, controle de concorrência e disponibilidade.

13.10. Domínio Próprio e HTTPS

O projeto foi publicado por meio do IP público da EC2. Em uma versão futura, seria ideal configurar um domínio próprio.

Além disso, a aplicação deveria utilizar HTTPS, garantindo comunicação criptografada entre usuário e servidor.

Essas melhorias tornariam a aplicação mais adequada para acesso público real.

13.11. Melhorias de Interface e Responsividade

Embora a interface atual atenda ao MVP, ela pode ser aprimorada para melhorar a experiência do usuário em diferentes dispositivos.

Possíveis melhorias incluem layout específico para celulares, melhor adaptação para tablets, página de detalhes do produto, melhorias visuais no carrinho, mensagens de confirmação mais claras e feedback visual após cadastro, edição ou exclusão.

Essas melhorias aumentariam a usabilidade e a qualidade percebida do sistema.

13.12. Testes Automatizados

Os testes realizados no projeto foram manuais. Em versões futuras, poderiam ser adicionados testes automatizados para garantir maior confiabilidade.

Esses testes poderiam incluir testes unitários no back-end, testes de integração da API, testes de componentes no front-end e testes de fluxo do usuário.

Com isso, futuras alterações poderiam ser feitas com menor risco de quebrar funcionalidades existentes.

13.13. Considerações sobre as Melhorias Futuras

As melhorias futuras propostas representam a evolução natural do MVP para um sistema mais completo e próximo de um e-commerce real.

O projeto atual fornece uma base funcional, com catálogo, carrinho local, área administrativa simples, API REST, banco de dados e deploy em nuvem. A partir dessa base, é possível evoluir gradualmente para um sistema mais seguro, escalável e completo.

14. Conclusão

O desenvolvimento do E-Commerce TDV permitiu a construção de uma aplicação web full-stack voltada à apresentação e ao gerenciamento de produtos elétricos. O projeto partiu da necessidade de organizar digitalmente o catálogo da TDV Elétrica e evoluiu para um MVP funcional, integrado e publicado em ambiente de nuvem.

A aplicação foi estruturada com separação entre front-end, back-end e banco de dados. O back-end foi desenvolvido em Python com FastAPI, utilizando SQLAlchemy para comunicação com o banco SQLite e Pydantic para validação dos dados. O front-end foi desenvolvido com React e Vite, permitindo uma interface dinâmica para visualização de produtos, busca, filtro por categoria, carrinho local e área administrativa simples.

Entre as funcionalidades implementadas, destacam-se o catálogo de produtos, a consulta de categorias, o uso de imagens reais, o carrinho persistido no navegador com `localStorage`, o login simples de administrador e as operações de cadastro, edição e exclusão de produtos. Essas funcionalidades atenderam ao escopo principal do MVP e permitiram demonstrar o funcionamento básico de uma loja virtual.

O projeto também contemplou a publicação em nuvem por meio do AWS Academy Learner Lab. A aplicação foi implantada em uma instância EC2 Ubuntu, com o back-end executado por Uvicorn e mantido como serviço com `systemd`, enquanto o front-end foi buildado com Vite e servido pelo Nginx. Essa etapa permitiu validar a aplicação fora do ambiente local e compreender aspectos práticos de infraestrutura, rede e deploy.

Durante o desenvolvimento, algumas funcionalidades inicialmente planejadas foram mantidas como melhorias futuras, como checkout real, autenticação segura, criptografia de senhas, geração de pedidos, pagamento online e histórico de compras. Essa decisão permitiu concentrar o desenvolvimento em uma entrega funcional e demonstrável dentro do escopo acadêmico.

Dessa forma, o projeto cumpriu seu objetivo principal ao entregar uma base funcional de e-commerce para materiais elétricos, aplicando conceitos de engenharia de software, desenvolvimento web, integração entre sistemas, versionamento e implantação em nuvem. A solução desenvolvida representa uma primeira versão do sistema, com estrutura preparada para evoluções futuras e melhorias incrementais.

15. Referências Técnicas

As referências técnicas utilizadas no desenvolvimento do projeto foram compostas principalmente pelas documentações oficiais das tecnologias adotadas, materiais de apoio

da disciplina e recursos utilizados durante a implementação e implantação da aplicação.

15.1. Tecnologias de Back-end

- **Python:** linguagem de programação utilizada no desenvolvimento do back-end da aplicação.
- **FastAPI:** framework utilizado para criação da API REST do sistema, responsável pelos endpoints de produtos, categorias e usuários.
- **Uvicorn:** servidor ASGI utilizado para executar a aplicação FastAPI em ambiente local e em ambiente cloud.
- **SQLAlchemy:** biblioteca utilizada para integração entre o código Python e o banco de dados SQLite.
- **Pydantic:** biblioteca utilizada para validação dos dados recebidos e retornados pela API.
- **SQLite:** banco de dados utilizado para persistência dos dados do MVP.

15.2. Tecnologias de Front-end

- **React:** biblioteca JavaScript utilizada para criação da interface da aplicação, controle dos estados e renderização dinâmica dos componentes.
- **Vite:** ferramenta utilizada para criação, execução e build do projeto front-end em React.
- **JavaScript:** linguagem utilizada na lógica do front-end, consumo da API, controle do carrinho e interações da interface.
- **Tailwind CSS:** utilizado para estilização da interface, organização visual dos componentes, cores, espaçamentos e responsividade.
- **Lucide React:** biblioteca utilizada para inclusão de ícones na interface da aplicação.

15.3. Infraestrutura e Deploy

- **AWS Academy Learner Lab:** ambiente cloud utilizado para criação da instância EC2 e publicação da aplicação.

- **Amazon EC2:** serviço utilizado para hospedar a aplicação em uma máquina virtual Ubuntu.
- **Ubuntu Server:** sistema operacional utilizado na instância EC2.
- **Nginx:** servidor web utilizado para servir o front-end React após o build do Vite.
- **systemd:** ferramenta utilizada para manter o back-end FastAPI executando como serviço em segundo plano.
- **Git e GitHub:** ferramentas utilizadas para versionamento do código-fonte e envio do projeto para repositório remoto.

15.4. Materiais de Apoio

Além das documentações oficiais das tecnologias, foram utilizados os conhecimentos adquiridos na disciplina de **Laboratório de Engenharia de Software**, especialmente nos seguintes temas:

- Levantamento e documentação de requisitos.
- Organização de backlog.
- Modelagem de sistemas.
- Desenvolvimento full-stack.
- Integração entre front-end e back-end.
- Versionamento de código.
- Implantação de aplicações em ambiente cloud.

15.5. Considerações sobre as Referências

As tecnologias escolhidas foram selecionadas por sua adequação ao escopo do projeto, facilidade de uso em ambiente acadêmico e compatibilidade com uma aplicação full-stack moderna.

O conjunto formado por FastAPI, React, Vite, SQLite, Nginx e AWS EC2 permitiu desenvolver, integrar e publicar uma aplicação funcional, atendendo aos objetivos definidos para o MVP do E-Commerce TDV.